

本节内容

浮点数

加减运算

(舍入问题)

IEEE 754 定义的4种舍入模式

考试默认

Eg: 2017_43_(3)

舍入模式	特点
★就近舍入（类似“0舍1入”）	IEEE默认舍入模式。类似“0舍1入”。如果3个舍弃位刚好是100，则若1+23bit尾数的末位为0，直接截断多余位即可；若1+23bit尾数的末位为1，则截断多余位，并在“末位+1”
正向舍入（向 $+\infty$ 舍入）	对正数进位（向上取整），对负数截断多余尾数
负向舍入（向 $-\infty$ 舍入）	对负数进位（向下取整），对正数截断多余尾数
截断法（向0舍入）	直接截断多余尾数，不进位

以float型加减运算为例：

尾数仅保留 1(隐含)+23 = 24 bit。IEEE 754 规定尾数“对阶”时，至少额外保留3bit冗余位

```
#include <fenv.h>
```

```
// 1) 就近舍入（默认）
```

```
fesetround(FE_TONEAREST);
```

```
// 2) 正向舍入
```

```
fesetround(FE_UPWARD);
```

```
// 3) 负向舍入
```

```
fesetround(FE_DOWNWARD);
```

```
// 4) 向0舍入
```

```
fesetround(FE_TOWARDZERO);
```

咸鱼拓展（不考）
在C语言中，如何
切换IEEE 754 的几
种舍入方式

浮点数的加减运算（就近舍入）

```
int main() {  
    // 定义并初始化变量  
    float x = 16777216.0f; X=224  
    float y = 0.5f;  
    float z = x + y;  
  
    // 打印十进制值  
    printf("十进制值:\n");  
    printf("x = %.1f\n", x);  
    printf("y = %.1f\n", y);  
    printf("z = x + y = %.1f\n\n", z);  
}
```

十进制值：
x = 16777216.0
y = 0.5
z = x + y = 16777216.0

若舍弃位为 0XX，则直接截断多余位即可

32位二进制表示：

x: 0 10010111 000000000000000000000000
y: 0 01111111 000000000000000000000000
z: 0 10010111 000000000000000000000000

```
int main() {  
    // 定义并初始化变量  
    float x = 16777216.0f; X=224  
    float y = 1.5f;  
    float z = x + y;  
  
    // 打印十进制值  
    printf("十进制值:\n");  
    printf("x = %.1f\n", x);  
    printf("y = %.1f\n", y);  
    printf("z = x + y = %.1f\n\n", z);  
}
```

十进制值：
x = 16777216.0
y = 1.5
z = x + y = 16777218.0

若舍弃位为 1XX，且 XX 不全为 0，则截断多余位，并在 1+23bit 尾数的“末位+1”

32位二进制表示：

x: 0 10010111 000000000000000000000000
y: 0 01111111 100000000000000000000000
z: 0 10010111 000000000000000000000001

例1: 尾数的舍入处理 (0舍)

已知 float $X = 16777216 = 2^{24}$, $Y = 0.5$ 符合 IEEE 754 标准。分析计算机实现 float $Z = X + Y$ 的过程。

$X \rightarrow +1.0 \times 2^{24}$ 0 10010111 000000000000000000000000

$Y \rightarrow +1.0 \times 2^{-1}$ 0 01111110 000000000000000000000000

尾数高位隐含1

+1.000000000000000000000000

← X 尾数原码

+1.000000000000000000000000

← Y 尾数原码

① 对阶 (原则: 小阶向大阶靠齐) $X_{\text{阶}} - Y_{\text{阶}} = 25$, 说明 Y 的阶码更小, 应将 Y 的尾数右移 25 位, 阶码对齐

硬件实现原理: 原码转补码, 加/减后再转回原码

X 尾数不变 →

+1.000000000000000000000000

移出的bit至少额外保留3位

Y 尾数右移 25 位 →

+0.000000000000000000000000010 00000000

② 尾数加减 (原码的加减运算)

原码加法, 得 Z 尾数原码 →

+1.000000000000000000000000010

③ 尾数规格化

$Z_{\text{阶}} = \text{大阶} + 0$
 $= 10010111 + 0$
 $= 10010111$

Z 尾数原码 →

+1.000000000000000000000000010

尾数天然地已经规格化 (尾数不用左规、右规, 阶码不变)

④ 尾数的舍入处理

舍弃多余尾数 (0舍1入)

+1.000000000000000000000000

舍弃位为 0XX, 直接截断

⑤ 溢出判断 → 阶码不全为 1、也不全为 0, 说明未发生上溢、下溢

【符, 阶, 尾】 = 【0 10010111 000000000000000000000000】

Z = 16777216

例2：尾数的舍入处理（1入）

已知 float $X = 16777216 = 2^{24}$, $Y = 1.5$ 符合 IEEE 754 标准。分析计算机实现 float $Z = X + Y$ 的过程。

$X \rightarrow +1.0 \times 2^{24}$ 0 10010111 000000000000000000000000

$Y \rightarrow +1.1 \times 2^0$ 0 01111111 100000000000000000000000

尾数高位隐含1

+1.000000000000000000000000

← X 尾数原码

+1.100000000000000000000000

← Y 尾数原码

① 对阶（原则：小阶向大阶靠齐） $X_{\text{阶}} - Y_{\text{阶}} = 24$ ，说明 Y 的阶码更小，应将 Y 的尾数右移 24 位，阶码对齐

硬件实现原理：原码转补码，加/减后再转回原码

X 尾数不变 →

+1.000000000000000000000000

Y 尾数右移 24 位 →

+0.000000000000000000000000110 00000000

移出的 bit 至少额外保留 3 位

② 尾数加减（原码的加减运算）

原码加法，得 Z 尾数原码 →

+1.000000000000000000000000110

③ 尾数规格化

$Z_{\text{阶}} = \text{大阶} + 0$
 $= 10010111 + 0$
 $= 10010111$

Z 尾数原码 →

+1.000000000000000000000000110

尾数天然地已经规格化（尾数不用左规、右规，阶码不变）

④ 尾数的舍入处理

舍弃多余尾数（0 舍 1 入）

+1.000000000000000000000001

舍弃位为 1xx，且 xx 不全为 0，则“末位+1”

⑤ 溢出判断 → 阶码不全为 1、也不全为 0，说明未发生上溢、下溢

【符，阶，尾】 = 【0 10010111 000000000000000000000001】

Z = 16777218

浮点数的加减运算（就近舍入）

```
int main() {  
    // 定义并初始化变量  
    float x = 16777216.0f; X=224  
    float y = 1.0f;  
    float z = x + y;  
  
    // 打印十进制值  
    printf("十进制值:\n");  
    printf("x = %.1f\n", x);  
    printf("y = %.1f\n", y);  
    printf("z = x + y = %.1f\n\n", z);  
}
```

十进制值：
x = 16777216.0
y = 1.0
z = x + y = 16777216.0

若舍弃位为 100，且 1+23bit 尾数的末位为 0，则直接截断多余位即可

32位二进制表示：

x: 0 10010111 000000000000000000000000
y: 0 01111111 000000000000000000000000
z: 0 10010111 000000000000000000000000

```
int main() {  
    // 定义并初始化变量  
    float x = 16777218.0f; X=224+2  
    float y = 1.0f;  
    float z = x + y;  
  
    // 打印十进制值  
    printf("十进制值:\n");  
    printf("x = %.1f\n", x);  
    printf("y = %.1f\n", y);  
    printf("z = x + y = %.1f\n\n", z);  
}
```

十进制值：
x = 16777218.0
y = 1.0
z = x + y = 16777220.0

若舍弃位为 100，且 1+23bit 尾数的末位为 1，则截断多余位，并在“末位+1”

32位二进制表示：

x: 0 10010111 000000000000000000000001
y: 0 01111111 000000000000000000000000
z: 0 10010111 000000000000000000000010

例3：尾数的舍入处理（100特殊处理）

已知 float $X = 16777216 = 2^{24}$, $Y = 1.0$ 符合 IEEE 754 标准。分析计算机实现 float $Z = X + Y$ 的过程。

$X \rightarrow +1.0 \times 2^{24}$ 0 10010111 000000000000000000000000

$Y \rightarrow +1.0 \times 2^0$ 0 01111111 000000000000000000000000

尾数高位
隐含1

+1.000000000000000000000000

← X 尾数原码

+1.000000000000000000000000

← Y 尾数原码

① 对阶（原则：小阶向大阶靠齐） $X_{\text{阶}} - Y_{\text{阶}} = 24$ ，说明 Y 的阶码更小，应将 Y 的尾数右移 24 位，阶码对齐

硬件实现原理：原码转补码，加/减后再转回原码

X 尾数不变 →

+1.000000000000000000000000

移出的 bit 至少
额外保留 3 位

Y 尾数右移 24 位 →

+0.000000000000000000000000100 00000000

② 尾数加减（原码的加减运算）

原码加法，得 Z 尾数原码 →

+1.000000000000000000000000100

③ 尾数规格化

$Z_{\text{阶}} = \text{大阶} + 0$
 $= 10010111 + 0$
 $= 10010111$

Z 尾数原码 →

+1.000000000000000000000000100

尾数天然地已经规格化
（尾数不用左规、右规，阶码不变）

④ 尾数的舍入处理

舍弃多余尾数（100 特殊处理）

+1.000000000000000000000000

⑤ 溢出判断 → 阶码不全为 1、也不全为 0，说明未发生上溢、下溢

【符, 阶, 尾】 = 【0 10010111 000000000000000000000000】

Z = 16777216

若 1+23bit 尾数的
末位为 0，则直接
截断多余位即可

例4：尾数的舍入处理（100特殊处理）

已知 float $X = 16777218 = 2^{24} + 2$, $Y = 1.0$ 符合 IEEE 754 标准。分析计算机实现 float $Z = X + Y$ 的过程。

$X \rightarrow +1.0 \times 2^{24}$ 0 10010111 000000000000000000000001

$Y \rightarrow +1.0 \times 2^0$ 0 01111111 000000000000000000000000

尾数高位
隐含1

+1.000000000000000000000001

← X 尾数原码

+1.000000000000000000000000

← Y 尾数原码

① 对阶（原则：小阶向大阶靠齐） $X_{\text{阶}} - Y_{\text{阶}} = 24$ ，说明 Y 的阶码更小，应将 Y 的尾数右移 24 位，阶码对齐

硬件实现原理：原码转补码，加/减后再转回原码

X 尾数不变 →

+1.000000000000000000000001

移出的 bit 至少
额外保留 3 位

Y 尾数右移 24 位 →

+0.000000000000000000000000 100 00000000

② 尾数加减（原码的加减运算）

原码加法，得 Z 尾数原码 →

+1.000000000000000000000001 100

③ 尾数规格化

$Z_{\text{阶}} = \text{大阶} + 0$
 $= 10010111 + 0$
 $= 10010111$

Z 尾数原码 →

+1.000000000000000000000001 100

尾数天然地已经规格化
（尾数不用左规、右规，阶码不变）

④ 尾数的舍入处理

舍弃多余尾数（100 特殊处理）

+1.000000000000000000000010

⑤ 溢出判断 → 阶码不全为 1、也不全为 0，说明未发生上溢、下溢

【符, 阶, 尾】 = 【0 10010111 000000000000000000000010】

Z = 16777220

若 1+23 bit 尾数的末位为 1，则截断多余位，并在“末位+1”